

A Thermal-aware Optimization Framework for ReRAM-based Deep Neural Network Acceleration

Hyein Shin, Myeonggu Kang, and Lee-Sup Kim

School of Electrical Engineering, Korea Advanced Institute of Science and Technology
{hyeinshin,mgkang}@mvlsi.kaist.ac.kr, leesup@kaist.ac.kr

ABSTRACT

Resistive RAM (ReRAM) is widely regarded as a promising platform for deep neural network (DNN) acceleration. However, the ReRAM device suffers from severe thermal problems that degrade the lifetime and inference accuracy of the ReRAM-based DNN accelerator. To address the issues, we propose a thermal-aware optimization framework for accelerating DNN on ReRAM (TOPAR). TOPAR includes 3-stage offline thermal optimization and online thermal-aware error compensation. Offline thermal optimization consists of thermal-aware weight decomposition, thermal-aware column reordering, and fine-grained weight adjustment to reduce the temperature of the ReRAM-based DNN accelerator. For online thermal-aware error compensation, we compensate conductance change according to the temperature variation. With TOPAR, the endurance degradation due to temperature rise improves up to 2.39×, and inference accuracy is preserved without harming the performance of the ReRAM-based DNN accelerator.

1 INTRODUCTION

Processing in-memory (PIM) approaches have been emerged to address the memory wall problem of the conventional DNN accelerators. Among various platforms of PIM, ReRAM is widely regarded as a proper platform for PIM with its intrinsic characteristic of matrix-vector multiplication [2, 11, 18]. Owing to the current-sum manner of ReRAM arrays, ReRAM-based DNN accelerators achieve high performance and energy-efficient computation.

Despite the promising computation ability of ReRAM-based DNN accelerators, thermal issues of ReRAM hinder its practical usage. According to [16], the endurance of the ReRAM cell reduces to 0.026× as temperature increases from 300K to 380K. Also, the conductance of ReRAM is extremely sensitive to temperature, which incurs more than 90% inference accuracy degradation at high temperature [23]. Furthermore, according to our observation, the thermal problems get more severe in ReRAM with low cell resolution (1~4bit), which is regarded as a realistic ReRAM compared to ideal ReRAM with high cell resolution (7~8bit) [2, 3, 11, 18]. Therefore, thermal problems of ReRAM should be mitigated as endurance and inference accuracy are crucial factors for practical usage of the ReRAM-based DNN accelerator.

[16, 17] aimed to enhance the endurance of the ReRAM cell by reducing peak temperature. However, their target application was not DNN acceleration, so the proposed schemes are not applicable to ReRAM-based DNN accelerators properly. Meanwhile, [15, 23] prevented accuracy degradation caused by temperature increase. [15] proposed thermal-aware row adjustment which maps significant weights with large values to cold cells periodically, while [23] mitigated accuracy degradation with bitwidth downgrading. However, [15] is not feasible with large scale DNNs, and its periodical writing on ReRAM cell harms the endurance of cells. [23] proposed method for large scale DNNs, but bitwidth downgrading of [23] is not applicable to the practical ReRAM with low cell resolution (1~4bit). For the ReRAM with low cell resolution, severe accuracy degradation occurs since it approximates LSB stored in the ReRAM cell, and the error is amplified by shift and add (S&A) process. Furthermore, there have been no previous works optimizing endurance and inference accuracy both.

In this work, we propose a thermal-aware optimization framework, TOPAR, which aims to both improve the endurance and preserve the inference accuracy of the conventional ReRAM-based DNN accelerator. TOPAR consists of offline and online thermal optimization. We perform offline thermal optimization utilizing the structure of the conventional ReRAM-based DNN accelerator. With the individual weight-level and array-level thermal optimization, our framework reduces average temperature and temperature variance between arrays for endurance enhancement in offline without harming the performance of the accelerator. In online thermal optimization, we propose thermal-aware error compensation, which is effective on the ReRAM with practical cell resolution and large scale DNN models. The major contributions of this paper are summarized as follows:

- We present TOPAR, an offline and online thermal-aware optimization framework, to enhance endurance and prevent inference accuracy degradation of the conventional ReRAM-based DNN accelerators.
- We propose 3-stage offline thermal optimization including thermal-aware weight decomposition, thermal-aware column reordering, and fine-grained weight adjustment to reduce average temperature and temperature variance between ReRAM arrays when running DNNs.
- We present online error compensation to preserve inference accuracy regardless of variation on the conductance value of the ReRAM cell.
- We evaluate TOPAR with five modern DNN models. Evaluation results show that TOPAR successfully enhances endurance and preserves the inference accuracy of ReRAM-based DNN accelerators.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ICCAD '20, November 2–5, 2020, Virtual Event, USA

© 2020 Association for Computing Machinery.

ACM ISBN 978-1-4503-8026-3/20/11...\$15.00

<https://doi.org/10.1145/3400302.3415665>

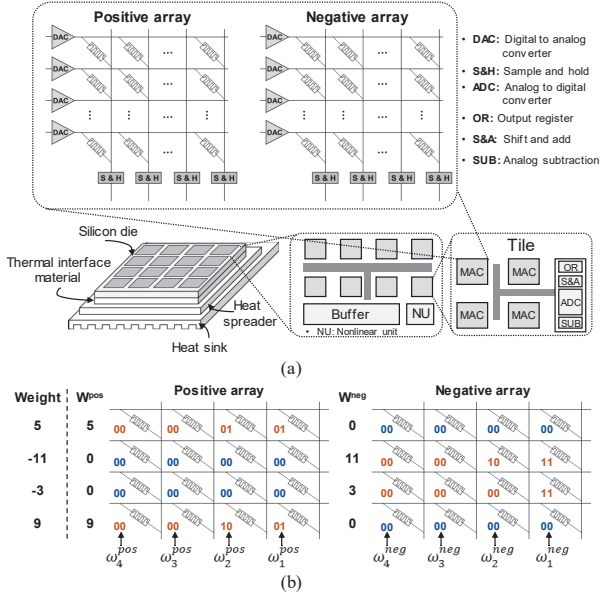


Figure 1: (a) The architecture of the conventional ReRAM-based DNN accelerator, (b) an example of 8bit quantized weights stored in ReRAM with 2bit cell resolution.

2 BACKGROUND AND MOTIVATION

2.1 ReRAM-based DNN accelerator

Architecture of the conventional ReRAM-based DNN accelerator is depicted in Fig. 1(a) [3, 18]. A heat sink, heat spreader, and thermal interface material compose a cooling system for the accelerator [9]. ReRAM arrays on the silicon die perform matrix-vector multiplication. Fig. 1(b) shows an example of weight allocation on ReRAM crossbar arrays. Since weights are signed values, a single weight is represented with both positive and negative cells (W^{pos} and W^{neg}) as conductance values shown in Fig. 1(b). For the positive weight, the absolute value of weight is stored in the positive array, and zero value is stored in the corresponding negative array. For the negative weight case, the absolute value of weight is stored in the negative array, and zero value is stored in the corresponding positive array.

Also, each W^{pos} and W^{neg} is split into multiple ReRAM cells as partial weights. As shown in Fig. 1(b), 8bit W^{pos} and W^{neg} are split into four 2bit ω_i^{pos} and ω_i^{neg} with 2bit cell resolution, where i indicates i^{th} partial weight. For matrix-vector multiplication between input features and weights, input features are applied to the word line through multiple time steps depending on the resolution of DAC. The multiplication results are obtained through each bit line as a current-sum manner. With the analog subtraction between current-sum results of the positive and negative array, partial sums are obtained and the results are converted to the digital domain via ADC as shown in Fig. 1(a).

2.2 Thermal problems of ReRAM-based DNN accelerator

The seriousness of thermal problems on the ReRAM has been raised recently [15–17, 23]. First, the endurance of the ReRAM cell decreases drastically with increased temperature. According to [16],

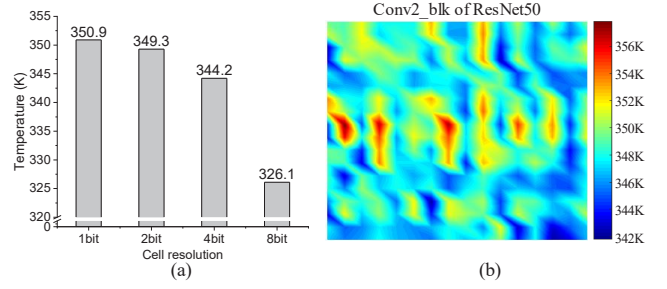


Figure 2: (a) Average temperature on various cell resolutions when running ResNet50 at 300K ambient temperature, (b) temperature distribution of ReRAM arrays when running conv2_blk of ResNet50.

the possible number of writes decreases from 4.14×10^8 to 1.07×10^7 when temperature increases from 300K to 380K, which highly reduces the lifetime of ReRAM arrays. Along with the degradation of endurance, temperature increase also changes the conductance value of the ReRAM cell. [15] stated that the ON state conductance drops by 50% while the OFF state conductance remains relatively unchanged. The variation of ON state conductance reduces possible conductance range to 50% as temperature increases from 300K to 400K, and the reduced conductance range incurs inaccurate computation result of DNN. According to [23], inference accuracy loss reaches up to 90% at high temperature. Furthermore, according to our observation, the temperature of ReRAM with practical cell resolution (1~4bit) rises higher, so it suffers from thermal problems more severely as shown in Fig. 2(a). The reason is that weights of DNN are densely distributed to small absolute values. For high cell resolution such as 8bit, an 8bit weight is allocated to a single ReRAM cell as a relatively small conductance value. However, in the case of low cell resolution, an 8bit weight is split and stored into multiple ReRAM cells, which is more prone to have large conductance value.

Fig. 2(b) shows an example of a huge temperature difference between arrays when running ResNet50 on ReRAM with 2bit cell resolution. The temperature difference reaches to 15.9K within a layer. Since the endurance is determined by the first failure time, an array with the peak temperature determines the lifetime of the ReRAM-based DNN accelerator [14, 16]. Therefore, it is important to reduce the average and peak temperature to enhance the lifetime of the ReRAM-based DNN accelerator. [16, 17] aimed to enhance endurance by reducing the peak temperature of ReRAM arrays, but their works are not applicable for DNN accelerator since they utilize the application-specific characteristics of caching or search-based PIM, respectively. Therefore, a novel method reducing average and temperature variance for the ReRAM-based DNN accelerator is required to enhance the lifetime of the accelerator.

In terms of accuracy, inaccurate computation results caused by conductance change should be compensated in addition to reducing temperature. [15] alleviated accuracy degradation due to temperature rise by mapping significant weights with large values to cold cells periodically. However, their work is not scalable to large scale DNN models as stated in [23]. [23] proposed a strategy for large scale DNNs, which discards the LSB stored in ReRAM cell at high temperature and shifts current-sum result 1bit left in the digital domain since conductance range of ReRAM cell varies from $0.5 \times$

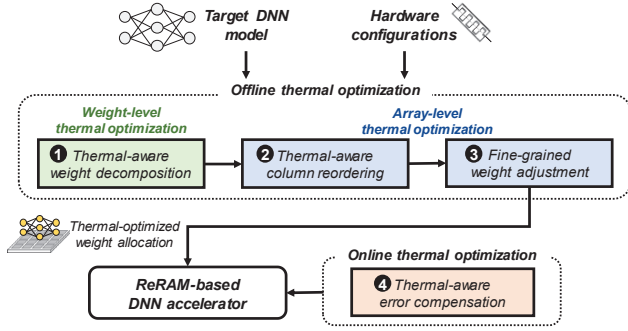


Figure 3: Overview of TOPAR.

to $1\times$. This method is effective at ReRAM with high cell resolution (7~8bit), but huge computation error occurs at practical ReRAM with low cell resolution (1~4bit). For example, with a 2bit cell resolution, data "11" stored in the ReRAM cell is changed to "10", and the difference between data "11" and "10" is amplified through the S&A process. Therefore, error compensation, which is also effective in low cell resolution and large scale DNNs, is required.

3 THERMAL-AWARE OPTIMIZATION FOR RERAM-BASED DNN ACCELERATION

We propose a thermal-aware optimization framework, TOPAR, to address the thermal problems of ReRAM-based DNN accelerators. As shown in Fig. 3, TOPAR consists of offline and online thermal optimization. TOPAR takes the target DNN model and hardware configurations, including cell resolution and array size as the inputs, and performs 3-stage offline and online thermal optimization. In the first stage of the offline optimization, ① thermal-aware weight decomposition is performed to reduce average temperature and temperature variance at the individual weight-level. The next two stages of offline thermal optimization, including thermal-aware column reordering and fine-grained weight adjustment, focus on reducing the temperature difference between arrays to enhance endurance and preserve inference accuracy further. As they aim to reduce the temperature difference between arrays, ② thermal-aware column reordering is performed based on a group of multiple arrays. ③ Fine-grained weight adjustment is performed within a pair of the positive and negative array for further alleviating the temperature difference between positive and negative array. Consequently, the target DNN model is allocated to ReRAM arrays with offline thermal optimization. While running the ReRAM-based DNN accelerator with allocated weights, online thermal optimization restores distorted current-sum results with the ④ thermal-aware error compensation to prevent inference accuracy degradation.

3.1 Thermal-aware weight decomposition

In the conventional ReRAM-based DNN accelerator, the absolute value of weight is stored in the positive or negative array according to the sign, and the value of zero is stored in the corresponding array as described in section 2.1 and Fig. 4(a). However, there are $2^N - V$ ways to decompose N bit value V into positive and negative arrays utilizing the structure of the conventional ReRAM-based DNN accelerator as shown in Fig. 4(b). When N is 8 and V is 15, weight 15 has $(256 - 15)$ possible decomposition cases.

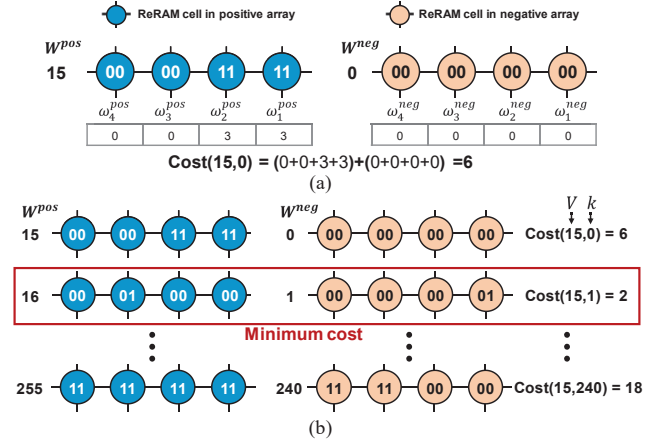


Figure 4: (a) The conventional method to store 8bit weight value 15 in positive and negative ReRAM arrays with 2bit cell resolution, (b) an example of thermal-aware weight decomposition when N is 8 and weight V is 15.

According to [6], the temperature increase is proportional to the power consumption of the block. In the case of ReRAM, since the sum of the partial weights stored in each cell is proportional to the power consumption, the temperature rise of positive and negative arrays can be modeled as Eq. 1:

$$\sum_{j=1}^{N_w} \sum_{i=1}^{N_c} (\omega_{j,i}^{pos} + \omega_{j,i}^{neg}) \propto \sum_{j=1}^{N_w} \sum_{i=1}^{N_c} (G_{j,i}^{pos} + G_{j,i}^{neg}) \propto (P_{pos} + P_{neg}) \propto \Delta T \quad (1)$$

where N_w indicates the number of weights stored in each ReRAM array and N_c indicates the number of cells required for representing single value [6, 12]. $\omega_{j,i}^{pos}$ and $\omega_{j,i}^{neg}$ represent i^{th} partial weight of j^{th} weight in positive and negative array, respectively. $G_{j,i}^{pos}$ and $G_{j,i}^{neg}$ indicate conductance of each ReRAM cell representing i^{th} partial weight of j^{th} weight which are proportional to partial weights $\omega_{j,i}^{pos}$ and $\omega_{j,i}^{neg}$. P_{pos} and P_{neg} represent power consumption of positive and negative array, and ΔT represents temperature rise of ReRAM arrays. Since the sum of partial weights are proportional to the temperature rise as described in the Eq. 1, our offline thermal optimization reduces average temperature and temperature variance by handling the partial weights stored in ReRAM arrays.

We propose a **thermal-aware weight decomposition** strategy which searches for a decomposition case with the smallest sum of partial weights among the possible decomposition cases ($2^N - V$ cases) to reduce the temperature of ReRAM-based DNN accelerator. First, we generate all possible decomposition cases and split each decomposition case into partial weights as shown in Fig. 4(b). In the next step, the sum of the partial weights for each decomposition case is calculated by a cost function as Eq. 2:

$$Cost(V, k) = \sum_{i=1}^{N_c} (\omega_i^{pos}(V, k) + \omega_i^{neg}(V, k)) \quad (2)$$

$$(W_{V,opt}^{pos}, W_{V,opt}^{neg}) = \arg \min_k Cost(V, k) \quad (3)$$

where V represents weight value and k indicates k^{th} decomposition case. $\omega_i^{pos}(V, k)$ and $\omega_i^{neg}(V, k)$ indicate i^{th} positive and negative partial weight of k^{th} decomposition case for weight V . As the cost function is set to the sum of partial weights, the smaller the cost value, the less the temperature rise based on Eq. 1. Therefore, we select the decomposition case with minimum cost, $(W_{V,opt}^{pos}, W_{V,opt}^{neg})$, among various decomposition cases of weight value V by following Eq. 3. Fig. 4(b) shows an example of thermal-aware weight decomposition for weight value 15. All possible decomposition cases representing value 15 are generated and the cost for each decomposition case, $Cost(15, k)$, is calculated. Based on the Eq. 2, the decomposition case (16,1) is obtained as the decomposition case with smallest cost, $(W_{15,opt}^{pos}, W_{15,opt}^{neg})$. When storing 16 to a positive array and 1 to a negative array to represent the weight value 15, the temperature rises the least while running DNN since the decomposition case (16,1) has the smallest sum of partial weights.

Thermal-aware weight decomposition does not change the exact value of weights in DNN models. It just modifies the way weights are allocated into positive and negative arrays, so retraining process is **not** required. For the weight value, which has multiple decomposition cases with the same minimum cost, we select a decomposition case with the smallest k . Decomposition cases with the same minimum cost are further explained in section 3.3.

3.2 Thermal-aware column reordering

As stated in section 2.2, temperature between arrays shows large difference. Therefore, array-level thermal optimization reducing the temperature difference between arrays is required. In the ReRAM-based DNN accelerator, weights of DNNs are allocated into ReRAM arrays as stated in [11]. Input channels are allocated into bit line direction, and output channels are allocated into word line direction across multiple ReRAM arrays as shown in Fig. 5(a). As each column of ReRAM array corresponds to output channels of DNN, shuffling the order of columns in ReRAM arrays changes the distribution of weights allocated in each array without harming the computation of DNN. Accordingly, we propose a **thermal-aware column reordering**, which reduces the temperature variance between arrays by shuffling the order of columns in ReRAM arrays.

For reordering columns, the architecture of the ReRAM-based DNN accelerator should be considered. ① Positive and negative array work as a pair for analog subtraction as described in section 2.1. Therefore, each column of the positive and negative array should be reordered together in pairs. ② A weight value is split into multiple partial weights according to the cell resolution and mapped into multiple columns. Since the partial weights are accumulated via the S&A process, the partial weights should also be reordered together. Therefore, we set the reordering pair as shown in Fig. 5(b). Note that the order of rows remains unchanged so that the order of channels in the input feature does not change.

To reduce the temperature difference between arrays by shuffling the order of pairs, reordering criteria should take into account the constraint that cells in positive and negative arrays within a pair are reordered together, but they are allocated into different arrays.

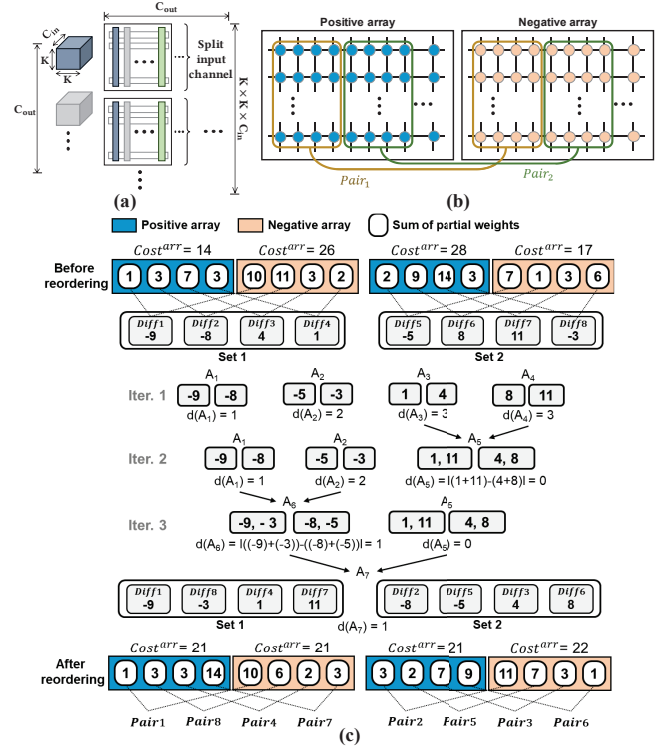


Figure 5: (a) Kernel mapping on ReRAM arrays, (b) reordering pairs in the case of 8bit weight with 2bit cell resolution, (c) an example of thermal-aware column reordering.

Therefore, we set the reordering criteria as Eq. 4:

$$Diff_i = \sum_{n=1}^{N_c} \omega_n^{pos} - \sum_{n=1}^{N_c} \omega_n^{neg} \quad (4)$$

where $Diff_i$ indicates difference between positive and negative array of i^{th} reordering pair, $Pair_i$, described in Fig. 5(c). N_c indicates the number of cells in a positive and negative array within a pair. ω_n^{pos} and ω_n^{neg} represent n^{th} partial weight in positive and negative array of each pair. With defining that the sum of the partial weights in each array as $Cost^{arr}$, we allocate each pair into arrays toward minimizing the difference of $Cost^{arr}$ based on the $Diff_i$ of pairs. Since we set the objective as minimizing the difference of $Cost^{arr}$, the column reordering is converted to an optimization problem, which is known as balanced multi-way number partitioning (BMNP) [7]. The BMNP is an optimization problem where each element is distributed to several sets, and the difference in the sum of each set is minimized. Among various solutions for the BMNP, the balanced largest first differencing method (BLDM) is regarded as the most effective solution for the BMNP [20]. Therefore, we aim to reduce the temperature variance of arrays by thermal-aware column reordering utilizing the BLDM. As the BLDM handles positive integer, we add bias term to make all $Diff_i$ values positive integer when implementing the BLDM.

Fig. 5(c) shows an example of the thermal-aware column reordering based on the BLDM. The example allocates eight pairs into two sets of arrays to minimize the difference of $Cost^{arr}$. A set includes

a pair of a positive and negative array. The difference of $Cost^{arr}$ reaches up to 14 before applying the column reordering. Initially, two $Diff_i$ values are grouped in ascending order as $A_1 = \{-9, -8\}$, $A_2 = \{-5, -3\}$, $A_3 = \{1, 4\}$, $A_4 = \{8, 11\}$. The first iteration replaces A_3 and A_4 to $A_5 = \{(1, 11), (4, 8)\}$ by grouping the smallest number of A_3 and A_4 with the largest number of A_4 and the rest since A_3 and A_4 have largest difference ($d(A_k), k=1, 2, \dots$) within each groups. In the next iteration, A_1 and A_2 are replaced by $A_6 = \{(-9, -3), (-8, -5)\}$. Finally, the result is obtained by replacing A_5 and A_6 as $A_7 = \{(-9, -3, 1, 11), (-8, -5, 4, 8)\}$ toward minimizing the difference of $Cost^{arr}$. By applying the thermal-aware column reordering based on the $Diff_i$ of pairs, the difference of $Cost^{arr}$ decreases to 1 consequently. Therefore, the temperature difference of arrays decreases with the thermal-aware column reordering.

To enlarge the effect of the column reordering, we reorder columns within a split input channel since the input channels are already split according to the size of the ReRAM array as shown in Fig. 5(a). As we reorder columns within a split input channel, the order of current-sum results should be unshuffled to the original order for obtaining input features for the next layer. However, huge performance loss and hardware overhead occur for unshuffling when we shuffle all pairs in the same split input channel. Therefore, we set reordering range as four sets of positive and negative arrays to avoid performance loss of the ReRAM-based DNN accelerator while maintaining the effectiveness of column reordering. With the reordering range, the additional 4:4 router routes current-sum result to output registers to revert the original order without harming the performance of the accelerator. Further analysis of hardware overhead for unshuffling is explained in section 4.6.

3.3 Fine-grained weight adjustment

With the thermal-aware weight decomposition (section 3.1), there are several decomposition cases having the same minimum cost. For example, weight value 10 has the same minimum cost when it is decomposed to either (10,0) or (16,6) as shown in Fig. 6(a). Although the cost is the same regardless of which one is selected at the individual weight-level, the two decomposition cases differ when considering the structure of ReRAM-based DNN accelerators. As weights are decomposed and stored into positive and negative array separately, the temperature rise of the positive and negative array varies depending on the selected decomposition case. With defining that the sum of the partial weights in the positive and negative array as $Cost^{pos}$ and $Cost^{neg}$, decomposition case (10,0) results in larger $Cost^{pos}$ than decomposition case (16,6) assuming the rest of the partial weights are the same as shown in Fig. 6(a). Therefore, we propose **fine-grained weight adjustment** to further reduce the temperature variance between positive and negative arrays by adjusting decomposition cases with the same minimum cost.

The fine-grained weight adjustment is performed upon the result of thermal-aware column reordering. Before adjusting decomposition cases, we first allocate weights which have a unique decomposition case with minimum cost into ReRAM cells since it is already thermally optimized. Next, we allocate the weights, which have more than two thermal-optimized decomposition cases. We select the decomposition case, which reduces the cost difference between positive and negative arrays most among the cases with

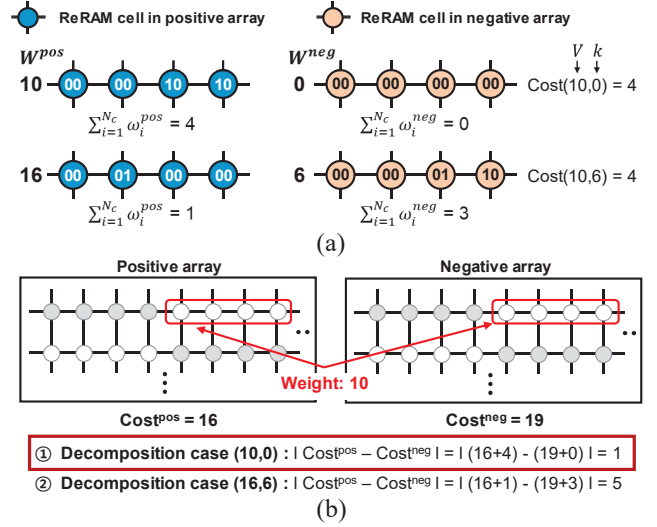


Figure 6: (a) An example of decomposition cases which have the same minimum cost for weight value 10, (b) an example of fine-grained weight adjustment.

the same minimum cost. Fig. 6(b) shows an example of allocating weight value 10, which has multiple thermal-optimized decomposition cases, into positive and negative array. Weights with a unique thermal-optimized decomposition case are first allocated into positive and negative arrays, which are represented as gray cells. In the case of storing the weight value 10, there are two thermal-optimized decompositions, (10,0) and (16,6). Among the two cases, the decomposition case, which further reduces the difference between $Cost^{pos}$ and $Cost^{neg}$, is selected. Assuming that current $Cost^{pos}$, the sum of partial weights represented as gray cells in the positive array, is 16 and $Cost^{neg}$, the sum of partial weights represented as gray cells in the negative array, is 19, decomposition case (10,0) is selected rather than (16,6) since decomposition case (10,0) reduces cost difference further as depicted in Fig. 6(b). Since fine-grained weight adjustment adjusts the thermal-optimized decomposition cases with the same minimum cost, temperature variance decreases while maintaining the average temperature as a minimum.

The fine-grained weight adjustment selects a suitable decomposition case based on the sum of partial weights mapped on an array. If there are several weights with multiple thermal-optimized decomposition cases in an array, they affect each other's weight adjustment. We perform the weight adjustment by following the greedy algorithm since the search space is large. The fine-grained weight adjustment is performed sequentially, starting with the weight mapped to the top-left position of the array.

3.4 Thermal-aware error compensation

Thermal-optimized weights are allocated to each ReRAM array physically with the offline thermal optimization described in Algorithm 1, and the ReRAM accelerator runs the DNN model online. Even though we reduce the temperature with the offline thermal optimization, temperature variation such as the rise of ambient temperature could hinder reliable computation of the ReRAM-based DNN accelerator. As ON state conductance varies from $0.5\times$ to $1\times$ at a temperature range between 300K~400K [15], the current-sum

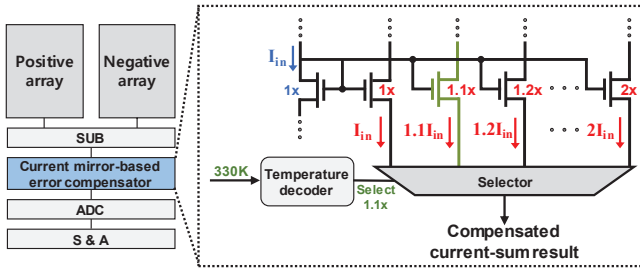
Algorithm 1: Pseudo-code of the offline thermal optimization

Input: DNN model, HW_config
Output: Thermal-optimized weight allocation

```

1: Initialize W_alloc(DNN model, HW_config) // Initialize weight allocation
2: foreach Layer  $\in$  DNN model do
3:   foreach Weight  $\in$  Layer do
4:     Update W_alloc  $\leftarrow$  W_decompose(Weight, HW_config)
      // Perform thermal-aware weight decomposition for each weight
5:   end
6:   Initialize R_groups  $\leftarrow$  Gen(W_alloc)
      // Generate reordering groups (Each R_group consists of 4 sets of pos&neg arrays)
7:   foreach R_group  $\in$  R_groups do
8:     Update W_alloc  $\leftarrow$  Col_reorder(R_group)
      // Perform thermal-aware column reordering within R_group
9:   end
10:  foreach (Array_pos, Array_neg) do
11:    Update W_alloc  $\leftarrow$  W_adjust(W_alloc, Weight, HW_config)
      // Perform fine-grained weight adjustment within a pair of pos&neg array
12:  end
13: end
14: Thermal-optimized weight allocation  $\leftarrow$  W_alloc

```

**Figure 7: Thermal-aware error compensation.**

result should be multiplied by 1~2 in order to restore the correct computation result. Also, since the results of the distorted current-sum are quantized to a digital value through the ADC, compensation in the digital domain cannot correct the error properly. Therefore, the error should be compensated in the analog domain.

For effective compensation, we propose **thermal-aware error compensation** by adopting a current mirror circuit, which shows reliable performance on analog multiplication regardless of temperature variation [24]. The current mirror multiplies the input current value by the ratio of the size of transistors as shown in Fig. 7. For accurate compensation, we set the size of transistors to be different by $0.1\times$ to support multiplication constants from 1 to 2 with 0.1 spacing. An example of thermal-aware error compensation is shown in Fig. 7. The thermal sensor network used in [15, 23] provides runtime temperature of each ReRAM array. Assuming the average temperature of the positive and negative array is measured as 330K by the thermal sensor, the temperature decoder converts the temperature information to select signal for compensation. Since ON state conductance changes to 0.9 times of original value at 330K, the current multiplied by 1.1 in the current-sum result is selected for compensation since 1.1 is reciprocal of 0.9. After the compensation, the restored current-sum results are converted to the digital domain via ADC. In addition, since we reduce the temperature variance between arrays with the offline thermal optimization, it is sufficient to add one compensation block per a pair of a positive and negative array after the analog subtraction. The current mirror-based error compensator is designed and

Table 1: Design configurations for evaluation

Design name	Configuration
Baseline	ReRAM-based DNN accelerator without thermal optimization
TOPAR-I	ReRAM-based DNN accelerator with thermal-aware weight decomposition
TOPAR-II	TOPAR-I with thermal-aware column reordering
TOPAR-III	TOPAR-II with fine-grained weight adjustment
TOPAR	TOPAR-III with online error compensation (All proposed schemes are included.)

verified by post-layout simulation under CMOS 65nm process with the process and temperature variation (300K~400K).

4 EVALUATION

4.1 Evaluation methodology

In this section, we evaluate TOPAR, a thermal-aware optimization framework, based on the practical configuration of ReRAM-based DNN accelerator with 2bit cell resolution and 64×64 array size [2, 12]. We use the ReRAM device parameter from the result of [13] operating at 1.2GHz. We perform thermal analysis based on the temperature of each array with validated thermal simulator Hotspot [9] assuming the ReRAM-based DNN accelerator is fully running. Detailed parameters of Hotspot including heat sink specification is set same as previous works [15, 16] and the ambient temperature is set to 300K. For peripheral circuit area, the resolution of ADC and DAC is set as 8bit and 1bit by following the result from [2].

We validate five modern DNN models using PyTorch framework [1], which are ResNet18, ResNet50 [8], VGG16 [10], stacked LSTM [21], and neural collaborative filtering (NCF) [22]. ResNet18, ResNet50, and VGG16 are evaluated with ImageNet [5]. 2-Layer stacked LSTM with 256 neurons is evaluated with WikiText-2 [19], and NCF for recommender system is evaluated with MovieLens [4]. For the five benchmarks, we use 8bit quantized input features and weights.

Table 1 shows the design configurations for evaluation. The baseline indicates the conventional ReRAM-based DNN accelerator without any thermal optimization. TOPAR-I represents the accelerator with thermal-aware weight decomposition. TOPAR-II represents TOPAR-I with thermal-aware column reordering, and TOPAR-III represents the ReRAM-based DNN accelerator with all offline thermal-aware optimization. Lastly, TOPAR represents the ReRAM-based DNN accelerator with all off/online thermal-aware optimization proposed in this work. For further validation of TOPAR, the ReRAM-based DNN accelerator with various cell resolutions [2, 3, 18] is also evaluated with ResNet50 in section 4.5.

4.2 Temperature reduction

Fig 8(a) shows average temperature running five benchmarks on the ReRAM-based DNN accelerator. We compare TOPAR-I to baseline on average temperature reduction since the thermal-aware column reordering and fine-grained weight adjustment focus on reducing the temperature variance between arrays. For the benchmarks, the average temperature decreases by 3.9K~7.4K compared to the baseline. The thermal-aware weight decomposition reduces average

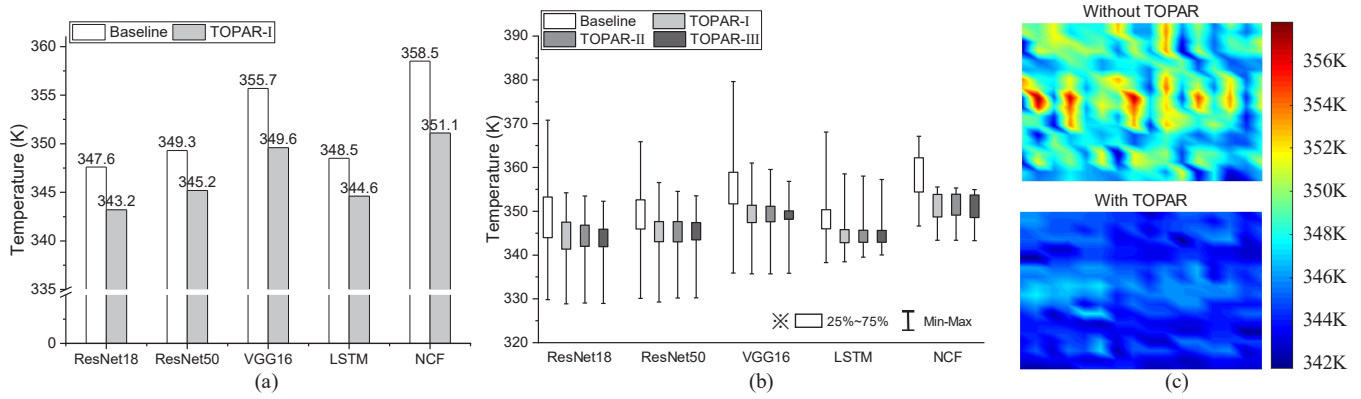


Figure 8: Comparison with baseline in terms of temperature (a) average temperature, (b) temperature variance between arrays, (c) thermal distributions of ReRAM arrays when running the conv2_blk of ResNet50.

temperature successfully by reducing the power consumption of array with minimized conductance values.

The effectiveness of TOPAR on reducing temperature variance between arrays is evaluated as shown in Fig. 8(b). We validate TOPAR-I,II,III for reduction of temperature variance since thermal-aware error compensation does not focus on temperature reduction. The box represents the range of 25%~75% of the total array temperature, and the line indicates the total temperature range of ReRAM arrays. As shown in Fig. 8(b), TOPAR-I not only reduces average temperature but also decreases temperature variance between arrays. In addition to TOPAR-I, TOPAR-II and TOPAR-III further reduce temperature variance between arrays. As shown in Fig. 8(b), our framework reduces temperature variance 34.9%~52.1% from the baseline. Peak temperature reduces the most for ResNet18 and VGG16, which have large temperature variance without thermal optimization. For NCF, the variance of 25%~75% of the total array temperature slightly increases with TOPAR-III, but overall variance represented as the total temperature range between minimum and maximum temperature decreases. Therefore, the temperature variance is also effectively decreased with our offline thermal optimization. Fig. 8(c) represents thermal distribution of ReRAM arrays running conv2_blk of ResNet50. As shown in the figure, overall temperature and temperature variance between arrays are effectively reduced by applying TOPAR. In addition, TOPAR can be combined with other dynamic thermal management techniques without harming the runtime performance of the ReRAM-based DNN accelerator since our thermal optimization is performed offline.

4.3 Endurance enhancement

Based on the temperature reduction shown in section 4.2, we evaluate endurance enhancement by TOPAR. Since previous works [16, 17] for enhancing endurance are not applicable to the ReRAM-based DNN accelerator, we compare our framework TOPAR with the baseline. For endurance evaluation, we use ReRAM with 4.14×10^8 possible number of writes at 300K and validate endurance enhancement in terms of the total amount of possible writes based on parameters of [16, 17]. Fig. 9 describes the total amount possible number of writes for the benchmarks normalized by the total amount of possible writes of baseline. Since the lifetime of ReRAM is defined as the first failure time with the highest temperature, we

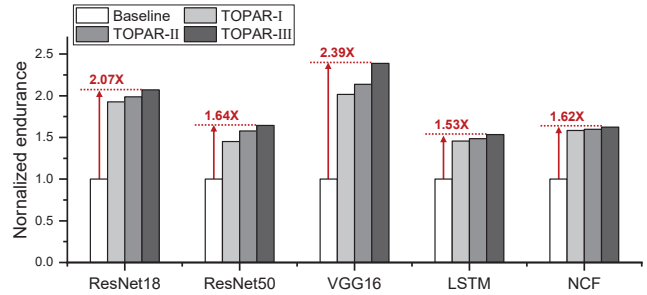


Figure 9: Endurance enhancement on 2bit cell resolution.

Table 2: Comparison on inference accuracy

Network	Top-1 Acc.	HR ³ AM [23]	TOPAR [†]	TOPAR
ResNet18	69.32%	0.25%	61.44%	69.28%
ResNet50	76.06%	1.79%	75.78%	76.1%
VGG16	71.46%	0.13%	70.53%	71.42%
Network	Perplexity	HR ³ AM [23]	TOPAR [†]	TOPAR
LSTM	106.73	648.43	108.8	106.89
Network	Hit ratio@10	HR ³ AM [23]	TOPAR [†]	TOPAR
NCF	0.6728	0.1939	0.6724	0.6728

[†] TOPAR without offline thermal optimization.

evaluate endurance enhancement with peak temperature running each benchmarks [14]. As stated in section 4.2, the improvement of endurance is validated with TOPAR-I, II, III since error compensation does not reduce temperature variance between arrays.

As shown in Fig. 9, ReRAM-based DNN accelerator with our offline thermal optimization shows $1.53\times \sim 2.39\times$ endurance enhancement. Along with the thermal-aware weight decomposition, the thermal-aware column reordering and fine-grained weight adjustment further improve the endurance of ReRAM arrays since peak temperature decreases with the reduced temperature variance between arrays.

4.4 Inference accuracy preservation

In this section, we compare our TOPAR to state-of-the-art design, HR³AM [23], on a practical ReRAM with 2bit cell resolution. For the accuracy evaluation, we build a simulator based on the Pytorch framework that supports the bit-level computation of the ReRAM accelerator, and reflect the change in the conductance of ReRAM

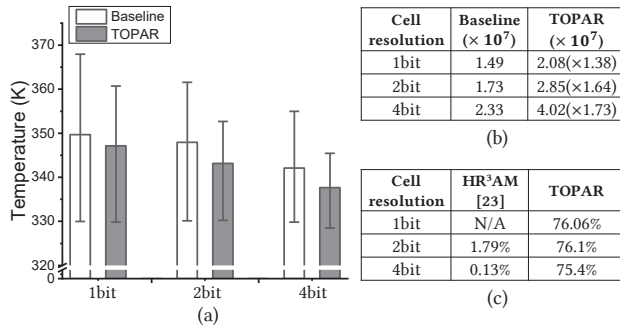


Figure 10: Evaluation on various cell resolutions when running ResNet50 (a) temperature reduction, (b) endurance enhancement, (c) accuracy preservation.

in the simulator according to the operating temperature. Note that TOPAR compensates distorted accumulation results from $1\times$ to $2\times$ with 0.1 spacing as we set the size of transistors to be different by $0.1\times$ described in section 3.4.

We compare TOPAR to HR³AM in terms of Top-1 accuracy for ResNet18/ResNet50/VGG16, perplexity (less is better) for LSTM, and hit ratio@10 for NCF. For each benchmark, the inference accuracy at 300K is shown in the second column of Table 2. Bitwidth downgrading of HR³AM [23] shows severe accuracy degradation since the error is amplified via the S&A process on ReRAM with low cell resolution. TOPAR†, TOPAR without offline thermal optimization, shows reasonable accuracy, but still does not reach the original accuracy. However, TOPAR, which includes off/online thermal-aware optimization, shows less than 1% accuracy degradation for all benchmarks. As shown in Table 2, reducing average temperature and temperature variance between arrays are also important for preserving the accuracy, although thermal-aware error compensation plays a major role. Also, setting the size of transistors for compensation as $0.1\times$ spacing is sufficient granularity as the degradation of accuracy, perplexity, and hit ratio is negligible.

4.5 Evaluation on various cell resolutions

The evaluation result for various cell resolutions [2, 3, 18] when running ResNet50 is shown in Fig. 10. The bar represents the average temperature, and the line indicates the total temperature range of ReRAM arrays. TOPAR effectively reduces average temperature and temperature variance of arrays on various cell resolutions as shown in Fig. 10(a). Based on the temperature reduction in Fig. 10(a), endurance enhancement and accuracy preservation are evaluated for ResNet50 with various cell resolutions. Fig. 10(b) represents the possible number of writes of the baseline and TOPAR. TOPAR improves the lifetime of the ReRAM-based DNN accelerator with various cell resolutions ranging from $1.38\times$ to $1.73\times$. In terms of accuracy preservation, TOPAR shows less than 1% accuracy degradation while HR³AM [23] shows severe accuracy loss with low cell resolution as shown in Fig. 10(c). Furthermore, HR³AM is not applicable to ReRAM with 1bit cell resolution since bitwidth cannot be reduced anymore.

4.6 Overhead analysis

Even though TOPAR aims to perform thermal optimization with least hardware modification on the conventional ReRAM-based

DNN accelerator, thermal-aware column reordering and error compensation require hardware modification. The hardware overhead for thermal-aware column reordering is modeled with a Verilog RTL, synthesized using Synopsys Design Compiler under CMOS 65nm library, and scaled to 32nm. The hardware overhead for thermal-aware error compensation is estimated by post-layout simulation under CMOS 65nm process, and also scaled to 32nm. In the case of thermal-optimized column reordering, an additional 4:4 router and registers for storing the original order of columns are required, and performance loss does not exist since we limit the reordering range. For the additional blocks, area and power overhead account for 0.012% and 0.021% of total ReRAM-based DNN accelerator, respectively. In the case of current mirror-based error compensation, area overhead is estimated to be 0.8% and power overhead is less than 0.001% of total ReRAM-based DNN accelerator. Consequently, the hardware modification required for TOPAR is negligibly small.

5 CONCLUSION

In this paper, we proposed the thermal-aware optimization framework for ReRAM-based DNN accelerators. We presented 3-stage offline thermal optimization and online error compensation to address the thermal issues of the ReRAM device. With our thermal optimization, average temperature and temperature variance of arrays decreased up to 7.4K and 52.1%, respectively. Owing to the temperature reduction, we improved the lifetime of the accelerator up to $2.39\times$. Also, inference accuracy had been preserved by our thermal-aware error compensation with less than 1% accuracy loss.

ACKNOWLEDGMENTS

This work was supported by the National Research Foundation of Korea(NRF) grant funded by the Korea government(MSIT) (No. NRF-2020R1A2B5B02002690).

REFERENCES

- [1] A. Paszke et al. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems* 32. 8024–8035.
- [2] A. Shafiee et al. 2016. ISAAC: A Convolutional Neural Network Accelerator with In-Situ Analog Arithmetic in Crossbars. In *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*. 14–26.
- [3] C. Xue et al. 2019. 24.1 A 1Mb Multibit ReRAM Computing-In-Memory Macro with 14.6ns Parallel MAC Computing Time for CNN Based AI Edge Processors. In *2019 IEEE International Solid-State Circuits Conference - (ISSCC)*. 388–390.
- [4] F. Harper et al. 2015. The MovieLens Datasets: History and Context. *ACM Trans. Interact. Intell. Syst.* 5, 4, Article 19 (Dec. 2015).
- [5] J. Deng et al. 2009. ImageNet: A Large-Scale Hierarchical Image Database. In *CVPR09*.
- [6] J. Lin et al. 2013. Thermal Modeling and Management of DRAM Systems. *IEEE Trans. Comput.* (2013), 2069–2082.
- [7] J. Zhang et al. 2011. Heuristic Algorithms for Balanced Multi-Way Number Partitioning. In *IJCAI*. 693–698.
- [8] K. He et al. 2015. Deep Residual Learning for Image Recognition. (2015). arXiv:1512.03385
- [9] K. Skadron et al. 2003. Temperature-aware microarchitecture. In *30th Annual International Symposium on Computer Architecture, 2003. Proceedings.* 2–13.
- [10] K. Simonyan et al. 2015. Very Deep Convolutional Networks for Large-Scale Image Recognition. In *International Conference on Learning Representations*.
- [11] L. Song et al. 2017. PipeLayer: A Pipelined ReRAM-Based Accelerator for Deep Learning. In *2017 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 541–552.
- [12] L. Xia et al. 2018. MNSIM: Simulation Platform for Memristor-Based Neuro-morphic Computing System. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 37, 5 (2018), 1009–1022.

- [13] M. Chang et al. 2014. 19.4 embedded 1Mb ReRAM in 28nm CMOS with 0.27-to-1V read using swing-sample-and-couple sense amplifier and self-boost-write-termination scheme. *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)* (2014), 332–333.
- [14] M. Murugan et al. 2011. Rejuvenator: A static wear leveling algorithm for NAND flash memory with minimized overhead. In *2011 IEEE 27th Symposium on Mass Storage Systems and Technologies (MSST)*. 1–12.
- [15] M. Valad Beigi et al. 2018. Thermal-Aware Optimizations of ReRAM-Based Neuromorphic Computing Systems (*DAC '18*).
- [16] M. Valad Beigi et al. 2018. THOR: Thermal-aware Optimizations for extending ReRAM Lifetime. In *2018 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 670–679.
- [17] M. Zhou et al. 2019. Thermal-Aware Design and Management for Search-Based In-Memory Acceleration (*DAC '19*).
- [18] P. Chi et al. 2016. PRIME: A Novel Processing-in-Memory Architecture for Neural Network Computation in ReRAM-Based Main Memory. In *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*. 27–39.
- [19] S. Merity et al. 2016. Pointer Sentinel Mixture Models. (2016). arXiv:1609.07843
- [20] W. Michiels et al. 2003. Performance Ratios for the Differencing Method Applied to the Balanced Number Partitioning Problem. In *Proceedings of the 20th Annual Symposium on Theoretical Aspects of Computer Science (STACS '03)*. 583–595.
- [21] Wo. Zaremba et al. 2014. Recurrent Neural Network Regularization. (2014). arXiv:1409.2329
- [22] X. He et al. 2017. Neural Collaborative Filtering. In *Proceedings of the 26th International Conference on World Wide Web (WWW '17)*. International World Wide Web Conferences Steering Committee, 173–182.
- [23] X. Liu et al. 2019. HR3AM: A Heat Resilient Design for RRAM-based Neuromorphic Computing. In *2019 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)*. 1–6.
- [24] F. Fiori and P. S. Crovetto. 2005. A new compact temperature-compensated CMOS current reference. *IEEE Transactions on Circuits and Systems II: Express Briefs* 52, 11 (2005), 724–728.